TestPilot AI

A QA Automation Platform, and the Application It Tests

Manual test design · API testing · UI automation · SQL validation · CI/CD · evidence-gated AI assistance

220	125	95.2%	10	5
automated tests	documented cases	of design automated	test suites	report formats

Python · pytest · Playwright · FastAPI · SQLite · GitHub Actions

Generated 2026-09-04

Contents

1. What this project is
2. Complete architecture
3. The test pyramid
4. Automation framework design
5. Playwright, explained
6. API testing, explained
7. SQL testing, explained
8. The CI/CD workflow
9. How to run the tests
10. How to demonstrate the project
11. The two-minute explanation
12. The five-minute explanation
13. Interview questions: QA fundamentals (15)
14. Interview questions: Selenium vs Playwright (8)
15. Interview questions: API testing (10)
16. Interview questions: SQL (11)
17. Interview questions: CI/CD (9)
18. Debugging scenarios (8)
19. Interview questions: AI in testing (5)
20. Resume bullets
21. Limitations and future improvements

Sections 13 to 19 hold 66 questions and answers in total, covering QA fundamentals, tooling, API and SQL testing, CI/CD, live debugging scenarios, and the use of AI in a test process.

1. What this project is

TestPilot AI is a QA automation platform, plus a sample ecommerce application called ShopNest that exists purely to be tested.

They are deliberately two separate systems. ShopNest is a FastAPI storefront with accounts, a catalogue, a cart, coupons and checkout, backed by SQLite and fronted by a browser UI. TestPilot never imports any of it. It reaches ShopNest only over HTTP and SQL, exactly as it would a service written by another team.

TestPilot does five things. It manages test cases as reviewable documents in YAML, each traced to the automated test that executes it. It runs suites through pytest and records every result into a relational store, with failure screenshots, Playwright traces and the HTTP exchange that produced the failure. It reports analytics that only exist because the results are relational: pass rate over time, genuine flakiness, failure hotspots. It tracks defects through a lifecycle enforced as a state machine. And it exports reports in five formats.

The part I would point at first is the AI layer, because of what it is not allowed to do. It can draft test cases from an OpenAPI spec, suggest edge cases, summarise a failure and classify it. What it cannot do is claim a bug exists. That is enforced in code: a defect claim is only admissible if a stored test result exists, its outcome is a failure, a traceback was captured, and the run is attributable to a build. A passing test can never produce a bug report. There are tests for that gate, and if they fail the AI layer is treated as unsafe to use.

The whole thing runs in CI on every pull request, and it is proven rather than assumed: there is a fault-injected build with three real defects deliberately planted in it, and the suite catches all three.

The test case library at a glance

Dimension	Breakdown
By layer	api: 82, db: 25, ui: 18
By type	boundary: 22, data: 18, functional: 33, integration: 1, negative: 28, security: 17, smoke: 6
By priority	P1: 60, P2: 51, P3: 13, P4: 1
Automated	119 of 125 (95.2%)

The six unautomated cases are documented with the reason, which is in every instance that the feature does not exist yet or the requirement is unresolved with product. They are not gaps that were quietly dropped.

2. Complete architecture

The repository holds two systems that share no code.

ShopNest is the system under test: a FastAPI storefront with authentication, a catalogue, a cart with coupons, checkout, and a SQLite database with real constraints — CHECK constraints on prices and quantities, UNIQUE on SKUs and emails, and foreign keys that are actually enforced at runtime. It serves a browser UI whose every interactive element carries a stable data-testid attribute.

TestPilot is the QA platform. It reaches ShopNest only over HTTP and SQL. Point TESTPILOT_BASE_URL somewhere else and it tests that instead.

The layering inside the automation is strict, and the rule is that a test may only call the layer immediately below it:

```
tests/ intent only - no selectors, no URLs, no SQL strings
framework/pages/ page objects: expose state, take actions, never assert
framework/clients/ API client and SQL client
framework/data/ builders and seeded constants
framework/utils/ waits, independent oracles, assertion helpers
conftest.py fixtures: server lifecycle, browser, contexts, capture
testpilot/ registry, store, runner, analytics, defects, AI
```

A test containing a CSS selector or a raw HTTP call has skipped a layer, and that is a review rejection.

Underneath, the platform is organised around one idea: execution produces rows, not logs. A pytest plugin records each run into test_runs and each test into test_results, carrying outcome, duration, markers, traceback, screenshot path, trace path, the HTTP conversation and the manual case id it traces to. Once that is relational, questions that are normally log-scraping exercises become one query each.

3. The test pyramid

The pyramid on this project is not a diagram in a slide deck; it is the actual distribution of tests, and it is enforced by a rule.

```
UI 18 tests ~30s journeys only a browser can prove
SQL / data 25 tests ~10s what was actually stored
API 82 tests ~50s every permutation, edge and negative
Platform unit 54 tests ~5s the measuring instrument itself
```

The rule: push every test to the lowest layer that can prove the thing.

Worked example. "A coupon must discount the cart exactly once" could be tested through the browser. It is not. It is tested at the API layer, where it runs in milliseconds and cannot be broken by a rendering change, across several cart shapes including the multi-line cart that exposes per-line stacking. It is reconciled again in SQL, where a query checks that every stored order's total equals its own components. The browser tests one instance of it, purely to prove the number reaches the screen.

Each layer answers a different question, and the questions do not substitute for one another:

```
Platform unit - is the measuring instrument itself correct?
API - does the service honour its contract, including edges?
SQL - did it actually store what it said it did?
UI - can a human complete the journey in a real browser?
```

That last distinction is why the SQL layer is not folded into the API layer. An endpoint can return 201 Created and write a wrong row; only SQL notices.

The inverted pyramid — mostly UI tests — is the common failure mode. It is slow, it is flaky, and when it fails it tells you a journey broke without telling you where. The suite then gets ignored, which is worse than having no suite, because people believe it.

4. Automation framework design

Six principles, each with a consequence you can see in the code.

1. Page objects expose state; tests make assertions. No page object in

```
this repository contains an assert. add_to_cart does not check that
the item was added, because that is the test's job — and it is what
lets the same method serve the happy path, the out-of-stock
rejection, and the quantity-cap boundary test.
```

2. Locators are data-testid. Class names and visible text change every

```
time somebody edits copy. A test id is a contract with the
application, and changing one is a breaking change.
```

3. No sleeps. Waiting is always waiting for a condition. The application

```
sets data-ready on the body once its fetches have rendered, and the
framework waits on exactly that.
```

4. Independent oracles. Tests must not import the application's own

```
pricing function to check its arithmetic. framework/utils/helpers.py
reimplements the documented rules, so a change in the application's
maths is caught rather than mirrored.
```

5. Tests are order-independent. Every test cleans up after itself and

```
gets a fresh browser context. Fixtures clear the cart on entry and
on exit. Any test that requires a predecessor is a defect in the
suite.
```

6. Failures must be self-explanatory. A bare assert response.status ==

```
200 produces a report nobody can act on. The assert_status helper
includes the response body, because that is what tells you whether it
was validation, authorisation or a genuine server error.
```

On fixture scope, which is where most suites go wrong: the application server and the browser are session-scoped because they are expensive. Everything else is function-scoped. Nothing is module-scoped — module scope makes tests share state within a file, and the resulting failures depend on which tests you happened to select.

The API client deserves a note. It never asserts; it reports what happened and lets the test decide what is correct, so one client serves a test expecting 201 and a test expecting 409. It records every request and response, which the fixture attaches to failing results. And it redacts credentials before anything is logged.

5. Playwright, explained

Playwright drives a real Chromium through the DevTools protocol rather than through the WebDriver HTTP protocol Selenium uses. Three consequences matter in practice.

Auto-waiting. Before acting on an element, Playwright checks that it is attached, visible, stable, enabled and able to receive events, and retries until it is or the timeout expires. Most of the explicit-wait scaffolding a Selenium suite accumulates simply does not need writing.

Browser contexts. A context is an isolated profile — its own cookies, its own localStorage — and creating one takes milliseconds, where launching a browser takes seconds. This suite launches one browser per session and creates one context per test. That is what gives every test a genuinely clean session cheaply, and it is why the suite can run in any order.

Tracing. This is the feature I would demonstrate first. A trace is a recorded timeline of the whole test: every action, a DOM snapshot before and after each one, network requests, console output and screenshots. Open it with `playwright show-trace artifacts/traces/<file>.zip` and you can step through the failure and inspect the live DOM at the moment it broke. Debugging a CI-only failure without one usually means adding print statements and pushing again.

The policy here is retain-on-failure: tracing starts for every test but the trace is only kept when the test fails. Full timelines for anything broken, without filling CI storage with traces of passing tests.

Even with auto-waiting, this suite adds its own readiness signal. The application sets data-ready="true" on the body once its initial fetches have rendered. Playwright's auto-waiting knows an element is actionable; it cannot know the page has finished loading its data. Waiting on the application's own signal beats waiting for network-idle, which is a guess about the network.

Other things used here: locators are lazy and re-resolve on each use, so they do not go stale; strict mode fails loudly when a locator matches more than one element rather than silently taking the first; wait_for_function is used where the assertion is about a computed DOM state, such as the cart badge reaching a value.

6. API testing, explained

The API layer carries 82 of the 125 documented cases, because it is the fastest and most stable place to test behaviour exhaustively.

The client gives tests intent-level methods — add_to_cart(product_id=3, quantity=2) rather than a hand-built POST — and handles three things tests should not each repeat: it never asserts, it records the whole request and response conversation for failure reports, and it redacts credentials.

What is actually covered:

Functional. Every documented behaviour, asserted on status code, body schema and business outcome.

Negative. 28 cases. Malformed emails in eight shapes, five weak-password variants, five malformed token shapes, SQL metacharacters, markup in search terms. Every negative test asserts both the rejection and that nothing changed — a 422 that still created a row is a worse defect than a 500.

Boundary. 22 cases, and the highest-yield category. Every documented limit is tested at the bound and one step either side. Cart quantity documented as 1..10 is tested at 0, 1, 2, 5, 9, 10, 11 and 9999. The free-shipping threshold is tested at exactly 5000 cents and at 4999. A coupon's minimum spend is tested at exactly the minimum and below it.

Authentication and authorisation. Valid credentials, wrong password, deactivated account, no header, wrong scheme, malformed token, a token signed with the wrong secret, an expired token, a customer attempting an admin action, and — the one people forget — a customer requesting another customer's order by id. That last is object-level authorisation, the classic IDOR, and it returns 404 rather than 403 so the endpoint does not confirm the record exists.

One case worth calling out: the login endpoint must return an identical message for an unknown email and a wrong password. If they differ, the endpoint is a free user-enumeration oracle. The test compares the two responses directly rather than checking each against a constant.

7. SQL testing, explained

The SQL layer answers the question the API cannot: not what did the service say, but what did it actually store.

Three kinds of check.

Schema contract. Every expected table exists. Every column the application reads or writes exists. Foreign keys are declared between related tables. The columns filtered on every request are indexed. And one that has saved real projects: every monetary and quantity column is declared INTEGER, because money is stored in integer cents and a schema drift to REAL introduces rounding error that is expensive to reconcile later.

Constraint enforcement. The CHECK constraints are real, not documentation. Inserting a negative price raises. A duplicate SKU raises. An order status outside the documented set raises. A cart item pointing at a nonexistent product raises — which also proves foreign keys are enforced at runtime, not merely declared.

Reconciliation. This is where the layer earns its place:

```
SELECT id, order_number, subtotal_cents, discount_cents,
shipping_cents, tax_cents, total_cents
FROM orders
WHERE total_cents <> subtotal_cents - discount_cents
+ shipping_cents + tax_cents;
```

That runs across every row and must return nothing. Similar queries check that every line total equals quantity times unit price, that every order subtotal equals the sum of its lines, that no order item is orphaned, that no product has negative stock, and that no two users share an email case-insensitively.

Two security checks live here too. No password may be stored as anything other than a PBKDF2 hash. And the count of distinct password hashes must equal the count of users — if two users share a hash, the salt is missing or constant.

There is also a reporting join, revenue by category across order_items, orders and products, cross-checked against the raw sum it is derived from. Aggregates are where a silently wrong JOIN first shows up, so the report is verified against the thing it summarises.

8. The CI/CD workflow

The pipeline runs on every pull request, every push to main, and nightly against main. It is shaped as a staircase: the cheapest checks run first and stop the build early, and the expensive browser suite only runs once the fast layers are green.

```
1. platform the platform's own unit tests, including the evidence
gate. If the measuring instrument is broken, nothing
it later reports can be trusted, so this gates all.
2. smoke fast build acceptance. One path through every
critical capability.
3. api-and-data API and SQL suites, as a matrix so they report
independently.
ui the browser suite, in parallel with the above.
4. report triage and a pull request comment. Never gates.
```

Splitting the API and browser suites into separate jobs matters: a browser flake cannot then mask an API regression, and you can see at a glance which layer broke.

Artifacts. Every job uploads its reports. The browser job additionally uploads failure screenshots and Playwright traces, so a CI-only failure can be debugged by downloading the trace and opening it locally, rather than by adding print statements and pushing again.

The pull request comment aggregates every suite's JUnit XML into one table plus collapsible detail per failure, each tagged with the manual case id it traces to.

Two deliberate choices. Concurrency is set so a new push cancels the in-flight run for the same branch. And CI runs with TESTPILOT_AI=off, so the AI layer uses its deterministic rule-based provider — a missing or expired model key can never turn a build red. AI assists analysis; it is never load-bearing for the pipeline.

9. How to run the tests

Setup:

```
python -m venv .venv
.venv/Scripts/activate (Windows)
source .venv/bin/activate (macOS / Linux)
pip install -r requirements.txt
playwright install chromium
```

Check the machine is ready. This validates the case library, the suite definitions, the environment config, the results database, and that Chromium actually launches:

```
python -m testpilot.cli doctor
```

Run something. The suite starts and stops ShopNest itself, so there is no separate step to forget:

```
tp run smoke fast build acceptance, about 14 seconds
tp run regression everything, about two minutes
tp run api API layer only
tp run ui browser suite
tp run ui --trace on browser suite, always keep traces
```

Look at results:

```
tp show the latest run in detail
tp history --suite regression pass rate over time, flaky tests
tp compare <baseline> <candidate> what broke since main
tp report <run> --format html export html/json/junit/csv/markdown
```

Triage and defects:

```
tp analyse <run> AI triage of the failures
tp defect file <result-id> draft and file, evidence permitting
tp defect move <id> TRIAGED --note "..."
tp defect list
```

Test design:

```
tp cases --layer api --detail 3
tp coverage
tp generate --spec http://127.0.0.1:8077/openapi.json --write
tp edges "cart quantity" --type integer
```

Run ShopNest on its own:

```
python -m uvicorn shopnest.main:app --port 8077

storefront http://127.0.0.1:8077/
API docs http://127.0.0.1:8077/docs
accounts casey@example.com / Custom3rPass
admin@shopnest.io / AdminPass123
```

Run TestPilot's own REST API:

```
python -m testpilot.api then http://127.0.0.1:8090/docs
```

10. How to demonstrate the project

A ten-minute demonstration that shows the discipline rather than the tool. Run it in this order; each step sets up the next.

1. Show the design before the code. (1 min)

```
tp coverage
tp cases --layer api --priority P1 --detail 2
```

125 documented cases, 119 automated, 95.2%. Point out that a case is a document with steps and an expected result, and that the coverage number measures coverage of that design rather than lines of code.

2. Run the smoke suite. (1 min)

```
tp run smoke
```

Eleven tests, about fourteen seconds, and note it reports itself as within its declared time budget. Say why smoke exists: it answers "is this build worth testing further" and nothing else.

3. Show a browser test actually running. (1 min)

```
TESTPILOT_ENV=headed tp run ui -- -k purchase
```

Chromium opens and drives the purchase journey slowly enough to watch. Mention that the test asserts against the database at the end — the UI said the order exists, SQL confirms it was stored.

4. The centrepiece: prove the suite catches real regressions. (3 min)

```
TESTPILOT_ENV=faulty tp run regression
```

164 passed, 5 failed. Three real defect classes were injected into that build — a coupon applied per line instead of per cart, an off-by-one allowing overselling, and an auth build that skips expiry checks and leaks other customers' orders. The suite caught all three. This is the difference between a suite that is green and a suite that is proven.

5. Triage the failures. (2 min)

```
tp analyse <run id>
```

Five failures, all classified product_defect, with expected and actual values quoted from the captured output. Point at the coupon one: expected 1199, actual 2396 — a customer was silently given double the advertised discount, which produces no error and is only noticed when revenue does not reconcile.

6. Show the evidence gate refusing. (1 min)

```
tp defect file <id of a PASSING test>
```

"Refusing to assert a defect. Test outcome was 'passed', not a failure." Then file from the real failure and show the drafted report: titled DRAFT, traced to manual case TC-CART-041, quoting the real assertion, with open questions for the reviewer.

7. Walk the lifecycle and show it refuse an illegal move. (1 min)

```
tp defect move <id> TRIAGED --note "Reproduced by hand."
tp defect move <id> IN_PROGRESS --note "..."
tp defect move <id> FIXED --note "..."
tp defect move <id> CLOSED <- refused
tp defect move <id> VERIFIED <- refused, needs a note
```

FIXED cannot jump to CLOSED. That single missing edge makes it structurally impossible to close a defect nobody re-tested.

8. Show the report and the artifacts. (1 min)

```
tp report <run> --format html
```

Open it: failure tracebacks, embedded screenshots, trace paths. If a UI test failed, open the trace with `playwright show-trace` and step through the DOM at the moment of failure.

If you only have two minutes, do steps 4, 5 and 6.

11. The two-minute explanation

For a recruiter or a first-round screen. What the project is, and the one thing about it worth remembering.

TestPilot AI is a QA automation platform, plus a sample ecommerce application called ShopNest that exists purely to be tested.

They are deliberately two separate systems. ShopNest is a FastAPI storefront with accounts, a catalogue, a cart, coupons and checkout, backed by SQLite and fronted by a browser UI. TestPilot never imports any of it. It reaches ShopNest only over HTTP and SQL, exactly as it would a service written by another team.

TestPilot does five things. It manages test cases as reviewable documents in YAML, each traced to the automated test that executes it. It runs suites through pytest and records every result into a relational store, with failure screenshots, Playwright traces and the HTTP exchange that produced the failure. It reports analytics that only exist because the results are relational: pass rate over time, genuine flakiness, failure hotspots. It tracks defects through a lifecycle enforced as a state machine. And it exports reports in five formats.

The part I would point at first is the AI layer, because of what it is not allowed to do. It can draft test cases from an OpenAPI spec, suggest edge cases, summarise a failure and classify it. What it cannot do is claim a bug exists. That is enforced in code: a defect claim is only admissible if a stored test result exists, its outcome is a failure, a traceback was captured, and the run is attributable to a build. A passing test can never produce a bug report. There are tests for that gate, and if they fail the AI layer is treated as unsafe to use.

The whole thing runs in CI on every pull request, and it is proven rather than assumed: there is a fault-injected build with three real defects deliberately planted in it, and the suite catches all three.

12. The five-minute explanation

For a technical interviewer. Leads with the reasoning behind each decision rather than the feature list.

Let me start with why the project is shaped the way it is, then walk the layers.

The problem with most QA portfolio projects is that they demonstrate a tool rather than a discipline. You see a Playwright script that logs into a site. What you cannot see is test design, traceability, whether the suite would actually catch a regression, or what happens when it fails at three in the morning. I built TestPilot to make those visible.

The two systems. ShopNest is the system under test: FastAPI, SQLite, a real browser UI. TestPilot is the QA platform. They share no code, and that is a substantive decision rather than tidiness. When I verify that a coupon discounts a cart correctly, my expected value is computed by an independent reimplementation of the pricing rules in the test framework. If I imported the application's own pricing function, a wrong rounding rule would be wrong identically on both sides and the suite would stay green. An oracle that shares code with the thing it checks proves nothing.

Test design comes first. There are 125 documented cases in YAML, each with an objective, preconditions, numbered steps, an expected result, a priority and a requirement trace. Only then is there automation, linked by an `automation:` field naming the pytest node. That link is what lets me report coverage of the *design* — 119 of 125 automated — rather than a raw test count. The six unautomated cases are documented with the reason, which is usually that the feature does not exist yet or the requirement is unresolved with product.

The pyramid is enforced by construction. 54 platform unit tests, 82 API tests, 25 SQL tests, 18 browser tests. The rule is to push every test to the lowest layer that can prove the thing. Coupon arithmetic is tested exhaustively at the API layer where it runs in milliseconds, and reconciled in SQL against the stored rows. Exactly one browser test covers it, purely to prove the number reaches the screen.

The SQL layer earns its place. An endpoint can return 201 Created and write the wrong row. So there are queries that reconcile every order: does the stored total equal subtotal minus discount plus shipping plus tax, across every row in the table. Does every line total equal quantity times unit price. Is any password stored in something other than a PBKDF2 hash. Those are questions the API physically cannot answer.

The UI layer is designed against flakiness. No sleeps anywhere. The application sets a `data-ready` attribute on the body when its fetches have rendered, and the framework waits on that — not on network-idle, which is a guess about the network, and not on a sleep, which is a guess about everything. Locators are all `data-testid`. The browser is session-scoped because launching Chromium is expensive; the browser *context* is function-scoped because contexts are cheap, and that is what gives every test a clean session in a few milliseconds and lets the suite run in any order.

Failures produce evidence, not just a red dot. When a browser test fails, a fixture screenshots the page and keeps the Playwright trace, which you can open with `playwright show-trace` and step through with a DOM snapshot at every action. When an API test fails, the client's recorded request and response conversation is attached. All of it lands in the results database against that result.

The AI layer, and the constraint on it. Four features: generate cases from an OpenAPI spec, suggest edge cases, summarise a failure, classify a failure into product defect, test defect, environment, test data or flake. Then a fifth that is gated twice: drafting a bug report. Gate one is admissible evidence — a stored

result, a failing outcome, a captured traceback, an attributable run. Gate two is that triage pointed at the product; a connection refused is not a product defect and will not be filed against the product. A human can override, and the override is recorded. Every drafted report is titled DRAFT, quotes only values actually present in the output, and carries an open-questions section.

There is a detail there I am fond of. The heuristic originally reported expected and actual reversed, because it scanned the traceback for status codes and assumed the first one found was the actual value — and a traceback quotes the assertion source *above* the rendered result. I fixed it to read pytest's own comparison line, and where neither pattern matches it now says the values are not determinable rather than guessing. Reporting expected and actual backwards is worse than reporting neither, and there are regression tests pinning that ordering.

Proving the suite works. A suite that has only ever been seen green is unproven. There is a fault profile that injects three real defects: a coupon applied per line instead of per cart, an off-by-one allowing overselling, and an auth build that skips expiry checks and leaks other customers' orders. Running regression against it fails exactly five tests, all correctly classified as product defects, with expected and actual values quoted correctly.

CI. GitHub Actions on every pull request, as a staircase: platform tests gate smoke, smoke gates the API, SQL and browser suites running in parallel. Screenshots and traces upload as artifacts, and a summary is posted as a PR comment. It runs with the AI in rule-based mode so a missing key can never redden a build.

What I would do next. Real concurrency testing for the checkout race I have documented but left manual, cross-browser coverage, and a proper visual regression layer.

13. Interview questions: QA fundamentals

Test design, process and judgement. The answers reference this project where it makes them concrete.

1. What is the difference between verification and validation?

Verification asks whether we are building the product right — does it match the specification. Reviews, static analysis and most automated tests are verification. Validation asks whether we are building the right product — does it solve the user's actual problem. Usability testing and acceptance testing are validation. In this project, the assertion that a coupon discounts 10% of the subtotal is verification; the open question about whether two coupons should stack (TC-CART-050) is a validation question, which is why it went back to product rather than being guessed at.

2. Explain the STLC phases.

Requirement analysis — decide what is testable and where the risk is; the useful output is usually questions. Test planning — scope, approach, environments, entry and exit criteria. Test case development — design cases and data, then automate. Test environment setup — somewhere reliable to run, and a way to prove it is ready. Test execution — run, record, report. Test closure — check exit criteria and review metrics. On this project each phase has an artefact you can open or run: the plan, the YAML case library, tp doctor, the results database, and tp history.

3. What is boundary value analysis and why does it find so many defects?

Boundaries are where conditional logic is written, so they are where off-by-one errors live. If a field is documented as 1 to 10 inclusive, you test 0, 1, 2, 9, 10 and 11 — the bound and one step either side. The reason it is high-yield is that >= versus > is a single character, it compiles either way, and it produces no error. This suite has 22 boundary cases. One of them, the free-shipping threshold tested at exactly 5000 cents and at 4999, is precisely the shape of defect that reaches production silently.

4. What is equivalence partitioning?

Divide the input domain into classes where every member should be treated identically, then test one representative of each rather than all of them. Cart quantity partitions into below-range, in-range and above-range. It is what keeps a suite finite. In practice you combine it with boundary analysis: partitioning tells you which classes exist, boundary analysis tells you to test at their edges.

5. What is the difference between smoke and regression testing?

Smoke answers one question — is this build worth testing further — and must be fast; here it is 11 tests covering one path through every critical capability, finishing in about 14 seconds. Regression asks whether anything that used to work has broken, and is broad; here it is the whole suite. Smoke gates regression in the CI pipeline, because running two minutes of tests against a build whose login is broken produces noise, not information.

6. What is a test case versus a test scenario?

A scenario is a high-level thing to be validated — 'a customer can check out'. A case is a specific, executable instance with concrete preconditions, steps, data and an expected result. One scenario yields many cases: checkout with one item, with a coupon, with insufficient stock, with an empty cart. In this repository the cases are the YAML documents, and each names the pytest node that executes it.

7. How do you decide what to automate?

Automate what is run more than twice, what guards money or security, and what a human would perform inconsistently. Do not automate exploratory testing, usability judgement, or one-off investigations. Automation is regression protection — it confirms known behaviour still holds. It does not find new classes of problem, and treating it as if it does is how teams stop doing exploratory testing.

8. What is severity versus priority?

Severity is how bad the damage is; priority is how soon we act. They are genuinely independent. A misspelled company name on the checkout button is S4 severity but likely P1 priority. A crash in a feature three users have is S1 but might be P3. Conflating them is how teams end up fixing crashes nobody hits while a typo damages the brand on the highest-traffic page.

9. What makes a good bug report?

Reproducibility above everything: exact steps, exact data, exact environment and build. Then expected versus actual, stated as values rather than adjectives. Then evidence — screenshot, trace, logs. Then an honest severity. The reports this platform drafts include all of that plus an open-questions section, because a report that admits what it does not know is more useful than one that overstates confidence.

10. What is a flaky test and how do you handle one?

A test that passes and fails without the code changing. It is worse than no test, because it trains people to re-run rather than investigate, and eventually a real regression gets re-run away. This platform defines flakiness precisely: a test that has both passed and failed in recorded history. A test that always fails is broken, not flaky, and is excluded from the flake report — conflating the two hides real regressions behind a 'known flake' label. The fix is to find the actual cause: usually a timing assumption, shared state between tests, or test order dependence.

11. How do you keep tests independent?

Every test sets up its own state and cleans up after itself. Here, the customer fixture clears the cart on entry and on exit, each UI test gets a fresh browser context, and tests that manipulate stock use a context manager that restores it. Nothing is module-scoped, because module scope makes tests share state within a file and produces failures that depend on which tests you selected. The test of independence is simple: the suite must pass when run in any order and when any single test is run alone.

12. What is traceability and why does it matter?

The ability to answer: which requirement does this test cover, and which tests cover this requirement. Here each YAML case carries a requirement id and names its pytest node, so coverage is reported against the documented design — 119 of 125 cases automated. It matters because without it you cannot answer 'have we tested the new discount rule' except by reading test names and hoping.

13. How do you test something with no requirements?

You write the requirements down as you discover the behaviour, and get them confirmed. Explore the system, document what it does, then take that back to product as 'this is what it currently does — is it intended?'. The output of testing an unspecified system is a specification. What you do not do is encode current behaviour as expected behaviour in a test, because then the suite locks in bugs.

14. What is the test pyramid and what happens when it is inverted?

Many fast unit tests, fewer integration tests, fewest UI tests. Inverted — mostly UI tests — you get a suite that is slow, flaky, and tells you a journey broke without telling you where. The real damage is social: the suite gets ignored, which is worse than having no suite, because people believe it. The governing rule is to push every test to the lowest layer that can prove the thing.

15. What is exploratory testing and does automation replace it?

Simultaneous learning, test design and execution — a skilled human investigating rather than following a script. Automation does not replace it and cannot: automation checks known expectations, and exploratory testing finds the expectations nobody thought to write down. They are complementary. Automation frees the time that makes exploratory testing possible.

14. Interview questions: Selenium vs Playwright

Tooling questions, answered from architecture rather than from a feature comparison table.

1. Selenium versus Playwright — what is the real difference?

Architecture. Selenium speaks the W3C WebDriver protocol over HTTP to a separate driver process, which speaks to the browser. Playwright speaks the browser's own DevTools protocol over a persistent WebSocket. That single difference produces most of the others: Playwright is faster because there is no HTTP round trip per command, it can intercept network traffic natively, and it can observe browser state that WebDriver does not expose. Selenium's advantage is maturity and reach: it is a W3C standard, supports far more browser and language combinations, and works with commercial grids everywhere.

2. How does auto-waiting work in Playwright, and what did Selenium make you write?

Before acting, Playwright checks that the element is attached, visible, stable, enabled and able to receive events, retrying until it is or the timeout expires. In Selenium you write that yourself with WebDriverWait and expected_conditions, and the common failure is reaching for implicit waits or sleeps instead. Auto-waiting removes most explicit-wait scaffolding, but it is not magic: it knows an element is actionable, not that the page has finished loading data. That is why this suite still adds an application-emitted data-ready signal.

3. What is a browser context and why does it matter for suite design?

An isolated profile — its own cookies, cache and localStorage — inside an already-running browser. Creating one takes milliseconds where launching a browser takes seconds. That changes suite architecture: you launch one browser per session and create one context per test, getting genuinely clean state per test almost for free. Selenium's equivalent is a whole new driver instance, which is expensive enough that people share sessions between tests and inherit order dependence.

4. What is Playwright tracing and when would you use it?

A recorded timeline of a test: every action, DOM snapshots before and after each, network activity, console output and screenshots. You open it with playwright show-trace and step through, inspecting the live DOM at any point. It is the answer to the hardest debugging problem there is — a test that fails only in CI. This project uses retain-on-failure: tracing runs for every test, traces are kept only for failures, and CI uploads them as artifacts.

5. Why not use XPath?

XPath couples the test to document structure, so wrapping an element in a div breaks it. Text-based selectors break when copy changes. CSS classes break when styling changes. This suite uses data-testid exclusively, which is a deliberate contract with the application: the attribute exists for the tests, changing it is a breaking change, and nothing else about the markup can break a locator. XPath is still worth knowing for traversal you genuinely cannot express otherwise, such as selecting a parent.

6. What is strict mode?

Playwright fails when a locator matches more than one element, instead of silently acting on the first. It converts a whole class of intermittent, wrong-element failures into an immediate, clear error that names how many elements matched. Selenium's find_element silently returns the first match, which is how a test ends up passing while clicking the wrong button.

7. Would you ever choose Selenium over Playwright today?

Yes, in three situations. If the organisation has a large existing Selenium estate and a working grid, the migration cost is real and the benefit is incremental. If you must support older browsers or an unusual browser that Playwright does not bundle. Or if you need a language binding Playwright does not have. For a greenfield suite on modern browsers, I would choose Playwright — mainly for tracing and contexts rather than for raw speed.

8. How do you handle a dynamic element that appears after an API call?

Wait for the condition, never for a duration. Playwright locators auto-wait for actionability, and for anything more specific you use wait_for_selector or wait_for_function with the actual predicate — this suite uses wait_for_function to wait for the cart badge to reach a value, for example. The anti-pattern is sleep(2), which is both slow when the element appears immediately and flaky when it takes longer.

15. Interview questions: API testing

REST semantics, authorisation, and how to cover an endpoint properly.

1. How do you test a REST API thoroughly?

Four axes. Happy path: correct request, correct status, correct body schema, correct business outcome. Negative: missing fields, wrong types, malformed payloads — asserting both the rejection and that nothing changed. Boundary: every documented limit at the bound and one step either side. Authorisation: no credential, bad credential, expired credential, valid credential belonging to the wrong user. That last one is the most commonly missed and the most damaging.

2. What status codes matter and how do you decide between them?

200 success, 201 created, 204 no content. 400 malformed request, 401 unauthenticated, 403 authenticated but not permitted, 404 not found, 409 conflict with current state, 422 well-formed but semantically invalid. The distinction that matters most: 401 means we do not know who you are, 403 means we do and you may not. And on this project, requesting another customer's order returns 404 rather than 403 — deliberately, because 403 would confirm the record exists.

3. What is idempotency and where does it matter?

An idempotent operation has the same effect whether applied once or many times. GET, PUT and DELETE should be idempotent; POST generally is not. It matters because networks retry. If a payment POST is not idempotent and the client retries after a timeout, you charge twice. This suite has an explicit case for it: cancelling an already-cancelled order must not credit stock a second time.

4. How do you test authentication?

Both directions, and the boundary between. Positive: valid credentials issue a token, and that token actually works on a protected endpoint — not just that it is well-formed. Negative: wrong password, unknown user, deactivated account, no header, wrong scheme, malformed token, a token signed with the wrong secret, an expired token. The expired-token test is worth designing carefully: mint it with the server's own secret, so expiry is the only possible reason for rejection.

5. What is broken object level authorisation and how do you test for it?

The endpoint checks that you are logged in but not that the object belongs to you, so changing an id in the URL returns somebody else's data. It is consistently near the top of the OWASP API list. You test it with two real accounts: A creates a resource, B requests it by id, and the response must not contain it. This suite tests it on both the detail endpoint and the list endpoint, because a tenant filter missing from a list query leaks everything at once.

6. How do you prevent user enumeration through a login endpoint?

Return an identical response for an unknown email and a wrong password, and take a similar amount of time over both. If they differ, an attacker can discover which addresses have accounts. The test compares the two responses to each other rather than each to a constant, which is what actually captures the requirement.

7. How do you test pagination?

Requested page size is honoured. The reported page count equals ceiling of total over page size. Pages do not overlap or skip — fetch two consecutive pages, confirm no shared ids, and confirm their concatenation matches the unpaginated order. A page beyond the last returns an empty list, not an error. And page size outside the documented range is rejected at 0, negative, and one over the maximum.

8. What is contract testing and how does it differ from what you did here?

Contract testing verifies that a provider's responses match an agreed schema, so a consumer can rely on shape without integration testing everything. Tools like Pact do it bidirectionally. What I did here is schema assertion inside integration tests — checking that expected fields exist and are consistent. The gap is real and I have listed it as a planned improvement: validating every response against the OpenAPI schema would catch a field that drifts even when no test asserted on it.

9. How do you handle test data for API tests?

Deterministic seeds for anything asserted on — known SKUs, known prices, exported as constants so a seed change breaks in one place. Generated unique data for anything that must not collide, so registration tests never fight each other. Builders rather than literal dicts, so a test states only the field it cares about and stays valid otherwise — which means it fails for the reason it claims to be about.

10. How would you test a rate limiter?

Send requests up to the limit and confirm they succeed; send one more and confirm 429 with a Retry-After header. Confirm the window resets. Confirm the limit is per the right key — per user, not global, or you have built a denial-of-service tool. And confirm rejected requests had no side effect. It is a boundary problem wearing a different hat.

16. Interview questions: SQL

Queries, constraints, transactions, and why the data layer is tested separately from the API.

1. Why test the database directly when you already test the API?

Because an endpoint can return 201 Created and write the wrong row. The API tells you what the service said; SQL tells you what it actually stored. On this project the SQL layer reconciles every order total against its own components, checks that line totals equal quantity times unit price, and confirms no password is stored as anything other than a PBKDF2 hash. None of those are answerable through the API.

2. Explain the JOIN types.

INNER returns rows matching in both tables. LEFT returns all rows from the left plus matches, with NULLs where there is none. RIGHT is the mirror. FULL OUTER returns everything from both. CROSS is the cartesian product. In testing, LEFT JOIN is the workhorse for finding orphans: left join children to parents and select where the parent id is NULL, and you have found every child pointing at nothing.

3. Write a query to find orphaned records.

SELECT oi.id, oi.order_id FROM order_items oi LEFT JOIN orders o ON o.id = oi.order_id WHERE o.id IS NULL. The LEFT JOIN keeps every order item; the IS NULL filter keeps only those whose parent lookup found nothing. That result set should always be empty, and this suite asserts exactly that.

4. What is the difference between WHERE and HAVING?

WHERE filters rows before grouping; HAVING filters groups after aggregation. You cannot use an aggregate in WHERE because the aggregate does not exist yet. Finding duplicate emails needs HAVING: GROUP BY LOWER(email) HAVING COUNT(*) > 1. Filtering to active users first would be WHERE.

5. How do you test that a transaction is atomic?

Force a failure partway and assert that nothing landed. This suite does it for checkout: build a cart, reduce stock below what the cart needs so the revalidation fails, attempt checkout, then assert three things — no order row was created, no stock moved, and the cart still holds its items so the customer can fix the problem. All three matter; checking only the order row would miss a stock decrement that leaked.

6. Why store money as integers?

Because binary floating point cannot represent most decimal fractions exactly, so 0.1 + 0.2 is not 0.3, and the error compounds across millions of rows until the ledger does not reconcile. Store cents as INTEGER, or use a DECIMAL type. This suite has a test asserting that every monetary column is declared INTEGER, specifically to catch a schema drift to REAL.

7. What is an index, and what is the cost?

A structure letting the engine find rows without scanning the table, usually a B-tree. The cost is write speed and storage: every INSERT, UPDATE and DELETE must maintain every index. So you index the columns you filter and join on, not everything. This suite asserts that products.category and orders.user_id are indexed, because those are filtered on every catalogue request and every order-history request.

8. What are ACID properties?

Atomicity — a transaction happens entirely or not at all. Consistency — it moves the database from one valid state to another, respecting constraints. Isolation — concurrent transactions do not observe each other's partial work. Durability — once committed, it survives a crash. The checkout in this project is the practical example: order header, line items, stock decrement and cart clear must all land or none of them.

9. How would you find duplicate rows?

GROUP BY the columns that should be unique and HAVING COUNT(*) > 1. For emails you group by LOWER(email), because 'Casey@example.com' and 'casey@example.com' are the same identity and a case-sensitive check would miss the duplicate — which is itself a defect worth testing for.

10. What is a foreign key and why check it at runtime?

A constraint that a column's value must exist in another table's key, which is what prevents orphans. The reason to test it rather than trust the schema is that enforcement can be off: SQLite requires PRAGMA foreign_keys = ON per connection, and it is silently off by default. A declared-but-unenforced foreign key is worse than none, because everyone assumes it is protecting them. This suite attempts an insert that violates it and asserts that it raises.

11. How do you validate an aggregate report?

Cross-check it against the raw data it derives from. This suite runs a revenue-by-category report joining three tables, then sums its own output and compares that with a direct sum of the underlying line totals. Aggregates are where a silently wrong JOIN first shows up — a duplicated join row inflates a SUM without any error — so the report is verified against the thing it summarises.

17. Interview questions: CI/CD

Pipeline design, gating, artifacts and secrets.

1. What is CI/CD and what does CI actually require of a test suite?

Continuous integration is merging to a shared branch frequently with automated verification on every change; continuous delivery keeps the result always releasable. What CI demands of a suite is specific: it must be fast enough that people wait for it, reliable enough that red means something, and self-contained enough to run on a clean machine. A slow or flaky suite does not slightly reduce the value of CI — it destroys it, because people start merging past it.

2. How is this pipeline structured, and why that way?

A staircase. Platform unit tests run first and gate everything, because if the measuring instrument is broken nothing it reports can be trusted. Then smoke, as build acceptance. Then API, SQL and browser suites in parallel. Then a reporting job that never gates. Splitting the API and browser suites into separate jobs means a browser flake cannot mask an API regression, and you can see at a glance which layer broke.

3. Which tests run on a pull request versus nightly?

Pull requests run everything except tests marked slow — the developer needs a complete answer before merging, and if the full suite is too slow for that, the suite is the problem. Nightly runs the full regression against main, and is where longer work belongs: cross-browser, performance, and anything too slow to gate a merge.

4. How do you debug a test that only fails in CI?

Get evidence out of CI rather than guessing. Screenshots and Playwright traces are uploaded as artifacts, so you download the trace and step through the failure locally with the DOM at each action. Then look for the usual causes: timing, because CI machines are slower — which is why the ci environment has longer timeouts; test order, if CI runs in a different order or in parallel; environment differences in timezone, locale or missing browser dependencies; and leftover state, since CI starts clean and your machine does not.

5. What should a pipeline do with test artifacts?

Always upload them, including on failure — an if: always() step, because the useful artifacts are precisely the ones from failed runs. Publish JUnit XML so the platform can annotate the pull request. Upload screenshots and traces for browser failures. And post a summary comment so a reviewer sees what broke without opening the run.

6. How do you handle secrets in CI?

Never in the repository. Injected as masked environment variables from the platform's secret store, scoped to the jobs that need them. In this project the design goes further: CI runs with the AI layer in rule-based mode, so the pipeline does not need a model key at all. A missing or expired key can never turn a build red. Test credentials in config/environments.yaml are seeded demo accounts for a local sample app, and real environments supply theirs from the process environment.

7. What is a quality gate and where should it sit?

An automated criterion that blocks a merge. Here: platform tests must pass, smoke must pass, and the layer suites must pass. Reporting deliberately does not gate — it informs. The judgement is that a gate should encode something the team genuinely will not ship without. Gating on a metric people can game, such as a coverage percentage, produces gaming rather than quality.

8. How do you keep a suite fast as it grows?

Keep the pyramid shape — the cheapest layer absorbs new cases. Give each suite a declared time budget and check it after every run, which this platform does: a smoke suite that has quietly grown to five minutes has stopped being a smoke suite. Parallelise across CI jobs and, where tests are genuinely independent, with pytest-xdist. And keep expensive setup at session scope while keeping state at function scope.

9. What does the shift-left idea mean in practice here?

Testing influences the product earlier than the test phase. Two concrete examples in this project. The data-testid attributes exist because testability was designed into the application rather than retrofitted. And the requirement questions raised during analysis — is the free-shipping threshold inclusive, does quantity zero mean remove — were answered before code was written, which is far cheaper than finding the ambiguity through a failing test later.

18. Debugging scenarios

Situational questions. Four of these are real failures encountered while building this project, with the actual root cause and fix.

1. A test passes locally and fails in CI. Walk me through it.

First get evidence rather than theorise: download the screenshot and the Playwright trace from the run's artifacts and step through the failure with the DOM at each action. Then work the four usual causes in order. Timing — CI machines are slower, which is why the ci environment here uses longer timeouts; look for any wait tied to a duration rather than a condition. Order and parallelism — try running the test alone and then in the CI order. Environment — timezone, locale, missing browser system dependencies. Leftover state — CI starts from a clean database and your machine has whatever previous runs left behind, which usually means the test depends on data it did not create.

2. The whole suite suddenly fails with 404s on every endpoint. What now?

Suspect the environment before the code, because a change that breaks every endpoint identically is rarely a code change. This actually happened during this project: an unrelated service was already listening on port 8000, the health check only asserted that something returned status ok, so the fixture attached to the wrong service and the entire suite ran against it. The fix was two-part — move to a less contested port, and make the health check assert the service *name*, so it now fails immediately with 'something other than ShopNest is listening on this URL' instead of producing a wall of confusing 404s.

3. Tests start timing out partway through a run, and everything after fails.

A progressive failure that starts mid-run points at resource exhaustion rather than at any individual test. Again, a real bug from this project: the fixture started the application server with stdout=subprocess.PIPE and nothing ever drained the pipe. Once the OS pipe buffer filled with log output, the server blocked on write and stopped answering — so the first N tests passed and everything after timed out. The fix was to redirect the server's output to a log file. The general lesson: when failures begin at an arbitrary point and never recover, look for something filling up.

4. A test fails intermittently, roughly one run in five. How do you diagnose it?

Do not re-run it until it passes. Check the recorded history first — this platform reports flake rate per test, so you can see whether it is genuinely intermittent or newly broken. Then look for the three classic causes: a timing assumption, usually a sleep or a wait on the wrong condition; shared state, where another test leaves data behind; and order dependence, which you test by running it alone and in different orders. If it is genuinely a product race, that is a real defect and the intermittency is the symptom, not the problem.

5. An API test fails with a 500. What information do you need?

The response body and the server-side traceback, which is why this framework attaches the full HTTP conversation to failing results and captures the server log. A 500 is the server's failure to handle something, so the interesting question is what input triggered it — which the recorded request tells you. In this project a real 500 was diagnosed exactly this way: the server log showed 'SQLite objects created in a thread can only be used in that same thread', because FastAPI resolves a request's dependencies across worker threads. The symptom was an intermittently blank page; the log named the cause immediately.

6. A UI test fails with 'element not found' but the element is clearly on the page.

Usually one of four things. The element is inside an iframe or shadow DOM and needs frame_locator. The locator matches several elements and strict mode is refusing to guess — the error message says how many. The page has not finished rendering, which is why this suite waits on an application-emitted data-ready flag rather than on the element alone. Or the element is present but not actionable — zero-size, covered by an overlay, or disabled. The trace answers all four immediately, because you can inspect the DOM at that exact moment.

7. You are told a suite has 95% code coverage but bugs keep reaching production.

Coverage measures which lines executed, not whether anything was asserted about them. A test that calls every function and asserts nothing scores 100%. I would look at what is being asserted, whether the edges are covered — coverage counts a branch as covered whether you tested the boundary or the middle — and whether the tests use independent oracles or just mirror the implementation. Then I would run mutation testing, which changes the code and checks whether any test notices; that is the honest measure. It is also why this project reports coverage of the documented test design rather than of lines: that number cannot be gamed by writing assertion-free tests.

8. A developer says your failing test is wrong. How do you handle it?

Treat it as a genuine possibility, because sometimes it is. Establish what the test asserts and what the documented requirement says. If they differ, the test is wrong and I fix it. If they agree, the product is wrong. If the requirement is ambiguous, neither of us is wrong and it goes to product — that is the TC-CART-050 coupon-stacking case in this project, which I left unautomated rather than encoding a guess. What I avoid is arguing from the test's authority; the requirement is the authority, and the test is just its executable form.

19. Interview questions: AI in testing

Where AI helps, where it is dangerous, and how this project constrains it.

1. Where does AI genuinely help in testing, and where is it dangerous?

It helps at the edges of the cycle: drafting cases from a specification, proposing edge cases a designer might not have considered, summarising a failure, and routing it to an owner. It is dangerous the moment it is allowed to assert something is true. A model asked 'is this a bug' will produce a confident, well-written answer whether or not any test failed, and a plausible bug report for a bug that does not exist costs more engineering time than no report at all — because someone has to disprove it.

2. How did you stop the AI claiming a bug exists?

By making it structurally impossible rather than asking it nicely. Every post-execution feature takes an Evidence object built from a stored test result, never free text. The gate admits a claim only when a stored result exists, its outcome is failed or error, a message or traceback was captured, and the run is attributable to a suite and environment. Bug reports need a second gate: triage must have pointed at the product, so a connection refused is never filed against the product. Both gates have their own tests, and if those go red the AI layer is treated as unsafe to use.

3. Why does the offline provider exist if you have model access?

Two reasons. CI must never fail because a key is missing or a service is slow, so the pipeline runs the deterministic provider by default — the AI layer assists analysis and is never load-bearing for the build. And a deterministic provider is testable: I can assert that a ConnectionError classifies as environment and not as a product defect. You cannot write that assertion against a model. The fallback is real analysis logic, not a stub.

4. How do you validate what a model returns?

Schema-validate everything and fall back rather than pass it through. A classification must be one of six known categories; anything else is rejected and the heuristic result is used. A generated case must have a title, a valid type, a valid priority and at least one step. A bug report must produce a known severity. The principle is that model output is an untrusted input to be validated like any other, not an answer to be displayed.

5. Would you let AI write your tests?

Draft them, yes; approve them, no. It is genuinely good at the mechanical breadth — given an endpoint with declared bounds, it will propose the boundary cases reliably, and that is real time saved. What it cannot do is know which requirement is commercially risky, which ambiguity needs a product decision, or that the coupon rule is the one worth testing five ways. In this project every generated case is marked status: draft and is excluded from every suite until a human reviews it.

20. Resume bullets

Written to lead with the outcome and name the technology second. Pick four or five; using all ten reads as padding.

- Built a full-stack QA automation platform (Python, pytest, Playwright, FastAPI, SQLite) covering 125 documented test cases with 95% automation, spanning API, SQL, browser and unit layers.
- Designed and enforced a test pyramid — 82 API, 25 SQL, 18 browser tests — reducing full regression runtime to under two minutes while keeping exhaustive negative, boundary and authorisation coverage.
- Implemented an evidence-gating layer that structurally prevents AI-assisted analysis from reporting a defect without a recorded failing execution, eliminating a whole class of false bug reports.
- Proved suite effectiveness with a fault-injection harness: three deliberately planted defect classes (per-line coupon stacking, stock off-by-one, missing token expiry) were all detected and correctly classified.
- Built a Playwright page-object framework with zero sleeps, using an application-emitted readiness signal and per-test browser contexts to keep the browser suite deterministic and order-independent.
- Automated failure diagnostics: full-page screenshots, Playwright traces retained on failure, and the captured HTTP request/response conversation attached to each failing result.
- Wrote SQL validation tests reconciling monetary totals, stock movements and referential integrity across every stored row, catching defects the API layer cannot surface.
- Delivered a GitHub Actions pipeline gating pull requests through staged suites, publishing JUnit reports, failure artifacts and an automated PR summary comment.
- Implemented a defect lifecycle as a validated state machine with an audit trail, structurally preventing a fix being closed without a recorded verification step.
- Built AI-assisted test design that drafts cases from an OpenAPI specification and proposes edge cases, with every output marked draft and gated behind human review.

A one-line summary, if you only have room for one

Built a QA automation platform covering API, SQL and browser layers with 125 documented test cases at 95% automation, including an evidence-gating mechanism that prevents AI-assisted analysis from reporting defects without execution proof.

21. Limitations and future improvements

Being able to name what a project does not do is worth more in an interview than claiming it does everything.

What this project does not do, and what I would build next.

Known limitations

Concurrency is documented but not executed. TC-ORD-040 describes two customers checking out the last unit simultaneously — the case that would genuinely exercise the transactional guard. It is written and left manual because doing it properly needs a concurrency harness rather than a sequential pytest run. The single-threaded stock revalidation is tested; the race is not.

SQLite is not the production database. It is excellent for a self-contained project, but its locking model differs from Postgres, so the transaction tests prove the logic rather than the behaviour under real contention.

Chromium only. The framework already supports Firefox and WebKit through TESTPILOT_BROWSER, but the suite is not run against them, so cross-browser rendering differences would not be caught.

No visual regression testing. A layout that breaks without changing any data-testid would pass every test in this suite.

No performance or load testing. There is no non-functional requirement stated for this release, and it needs a separate harness and a representative environment to mean anything.

Accessibility is documented, not automated. TC-UI-030 describes the keyboard-only purchase journey but remains manual until an axe-core audit is integrated.

The AI heuristics are pattern-based. The offline provider classifies failures using ordered regular expressions over the traceback. That is deterministic and explainable, which is why CI uses it, but a novel failure shape falls through to "undetermined" and needs a human. That is the intended behaviour rather than a bug, but it is a limit.

Test data is seeded, not production-shaped. Ten products and four accounts exercise the logic but would not surface pagination or query performance problems that only appear at scale.

Planned improvements

- A concurrency harness so TC-ORD-040 can be automated, running real

```
simultaneous checkouts and asserting exactly one succeeds.
```

- Postgres as a second target for the data layer, to test the

```
transactional behaviour the production database would actually have.
```

- Cross-browser execution in the nightly job, where the extra runtime

```
does not slow pull requests down.
```

- Visual regression on the three key pages, with a reviewable baseline. - Contract testing against the OpenAPI schema, so a response that drifts

```
from its declared shape fails even when no test asserted on that field.
```

- Mutation testing on the pricing module, which is the honest way to

```
find out whether these tests would actually catch a change.
```

- Flake quarantine: automatically de-gate a test whose flake rate

```
crosses a threshold, and open a defect against the suite itself.
```

The one thing to remember

A test suite that has only ever been seen green is an unproven suite. This one is run against a build with three deliberately injected defects, it catches all three, and the AI layer that reports them is structurally forbidden from claiming a defect exists without a recorded failing execution behind it.